



MODUL 8 · STATE MANAGEMENT

React Context: Kapan Sudah Cukup



Context adalah primitive React — tapi punya keterbatasan yang penting di App Router.

Context di App Router — Keterbatasan Penting

Context tidak bisa dipakai di Server Component. Harus dibungkus Provider di 'use client'.

✗ Tidak valid — Context di Server Component

```
// ✗ Server Component TIDAK BISA pakai Context
// Tidak ada hooks di Server Component!

export default async function ServerPage() {
  // Error: useContext tidak tersedia di server
  const { user } = useContext(UserContext)
  const { theme } = useContext(ThemeContext)

  // Server Component tidak punya "component tree"
  // runtime React — tidak ada context
  // propagation.
  // Context hidup di browser (hydrated tree).
  // Server render HTML statik — tidak ada
  // context.
  return <div>{user?.name}</div>
}

// Jika butuh user di Server Component:
// Baca dari cookies() / session, bukan Context!
```

✓ Context hanya di Client Components

```
// Provider harus di 'use client'
// app/providers.tsx
'use client'
import { createContext, useContext, useState } from 'react'

const ThemeContext = (createContext < 'light' > 'dark' > 'light')

export function ThemeProvider({ children }: { children: React.ReactNode }) {
  const [theme, setTheme] = (useState < 'light' > 'dark' > 'light')
  return <ThemeContext.Provider value={theme}>{children}</ThemeContext.Provider>
}

export const useTheme = () => useContext(ThemeContext)

// app/layout.tsx — wrap children
// (bukan Server Component karena ThemeProvider punya 'use client')
export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        <ThemeProvider>{children}</ThemeProvider>
      </body>
    </html>
  )
}
```

Children Server Component dalam Provider Client

Provider butuh 'use client' — tapi children bisa tetap Server Component. Ini valid!

Pattern yang valid: Provider wraps Server Component children

```
// app/providers.tsx
'use client'
export function Providers({ children }: { children: React.ReactNode }) {
  return (
    <QueryClientProvider client={queryClient}>
      <ThemeProvider>
        {children} {/* children bisa Server Component! */}
      </ThemeProvider>
    </QueryClientProvider>
  )
}

// app/layout.tsx – Server Component
import { Providers } from './providers'
import { UserProfile } from '@components/user-profile' // Server Component
export default function RootLayout({
  children,
}: {
  children: React.ReactNode,
}) {
  return (
    <html lang="id">
      <body>
        <Providers>
          {/* children = Server Component pages – valid! */}
          {/* Provider 'use client' tidak menginfeksi children */}
          {children}
        </Providers>
      </body>
    </html>
  )
}

// Kenapa ini valid?
// children di-PASS sebagai props dari layout.tsx (Server Component)
// Provider hanya menerima dan merender children-nya
// children TIDAK menjadi bagian dari Client Component tree Provider
```

Kapan Context Cukup vs Kapan Butuh Zustand?

Context sudah cukup untuk banyak kasus. Zustand diperlukan untuk performa dan kompleksitas.

Kriteria	React Context	Zustand
Re-render behaviour	Re-render SEMUA consumer ketika value berubah	Only re-render component yang subscribe field berubah
Update mechanism	setState di Provider — semua consumer ikut render	Selector-based: subscribe hanya field yang perlu
DevTools	Tidak ada built-in DevTools	Zustand DevTools via Redux DevTools Extension
Async actions	Manual handle async di dalam provider	Built-in dengan actions yang bisa async
Middleware	Tidak ada built-in	persist, immer, subscribeWithSelector, dll

Mengatasi Context Re-render Problem

Split context atau pakai useMemo untuk hindari unnecessary re-render.

Split Context — satu concern, satu context

```
// ❌ Satu context besar — semua consumer re-render saat apapun berubah
const AppContext = createContext({ user, cart, theme, notifications })

// ✅ Split per domain — consumer hanya re-render jika konteksnya
const UserContext = (createContext < User) | (null > null)
const CartContext = createContext < CartState > { items: [], total: 0 }
const ThemeContext = (createContext < 'light' > 'light') | ('dark' > 'light')

// Provider terpisah — compose di layout
function Providers({ children }: { children: React.ReactNode }) {
  return (
    <ThemeProvider>
      {' '}
      {/* ThemeContext.Provider */}
      <UserProvider>
        {' '}
        {/* UserContext.Provider */}
        <CartProvider>
          {' '}
          {/* CartContext.Provider */}
          {children}
        </CartProvider>
      </UserProvider>
    </ThemeProvider>
  )
}

// Consumer: hanya subscribe ke context yang dibutuhkan
function CartIcon() {
  const { items } = useContext(CartContext) // hanya CartContext
  // ✅ Tidak re-render saat user atau theme berubah
  return <span>{items.length}</span>
}
```


Evaluasi Pemahaman

Jawab sebelum lanjut ke slide berikutnya.

Q1 — Mengapa Context tidak bisa dipakai di Server Component?

- A) Server Components tidak support JSX
- B) Context bergantung pada React runtime (hooks) yang hanya tersedia di browser/Client Component
- C) Perlu install package tambahan untuk Server Component

Q2 — Provider butuh 'use client'. Apakah children yang dipass ke Provider menjadi Client Component?

- A) Ya — semua menjadi Client Component
- B) Tidak — children yang dipass sebagai props tetap Server Component
- C) Tergantung isi children

Q3 — Apa masalah utama Context saat state sering berubah di aplikasi besar?

- A) Memory leak karena Provider tidak di-cleanup
- B) Semua consumer re-render ketika value berubah — bisa jadi performance issue
- C) Context tidak bisa dipakai lebih dari 3 level

Tugas Mandiri

Implementasi Context untuk theme dan validasi limitasinya.

01 ThemeContext

Buat ThemeProvider dengan Context. Bungkus di layout.tsx. Buat useTheme hook. Pastikan Server Component children tidak terpengaruh.

02 Split Context

Buat UserContext dan CartContext terpisah. Composite di Providers wrapper. Verifikasi: update cart tidak re-render UserDisplay.

03 Test Re-render

Gunakan React DevTools Profiler. Record saat cart berubah. Berapa komponen yang re-render? Bandingkan satu context besar vs split.

04 Context vs Zustand

Buat dua implementasi CartContext: satu dengan React Context, satu dengan Zustand. Bandingkan boilerplate, DX, dan performa di DevTools.



Buka React DevTools → Profiler → Record → tambah item ke cart → lihat highlight komponen yang re-render.

Yang Sudah Kita Pelajari

- ✓ Context tidak bisa di-consume di Server Component — hanya tersedia di Client Components.
- ✓ Provider harus 'use client' tapi children yang dipass sebagai props tetap Server Component.
- ✓ Context re-render ALL consumers saat value berubah — split per domain untuk performa.
- ✓ Context cukup untuk: theme, locale, simple user session. Zustand untuk yang lebih kompleks.
- ✓ Rule: gunakan Context dulu, migrasi ke Zustand hanya saat ada masalah konkret.

→ Selanjutnya: **Bab 3 — Zustand: Store Factory + Integrasi Next.js**

